

Architecture Design Report

This document presents the architectural design of the system, including its context, logical structure, runtime behavior, and deployment.

1. System Requirements

System Name: LibraryManagementSystem

Architecture Style: Client Server

Functional Requirements

- The system must detect the user's face using the YOLOv5 model.
- The system must recognize gestures.
- The system must track the user's hand.
- The system must support single-hand interaction.
- The system must allow users to customize actions for specific gestures.
- The system must enable real-time collaboration.
- The system must have low-latency.

Non-Functional Requirements

- The system must be cost-effective.

Description: The system costs less than all other touch projectors in the market, enabling users to run the system on a budget.

- The system must be accessible to many users.

Description: The system is designed to be affordable and accessible to many users.

- The system must be flexible.

Description: The system can work effectively in different places.

- The system must be secure.

Description: The system stores account information of the user in an account database.

- The system must be user-friendly.

Description: The system provides an interactive surface controlled remotely with just hand gestures.

2. Formal Architecture Specification (ACME ADL)

The following section provides a formal architectural specification using ACME ADL, capturing components, connectors, and constraints.

```

System LibraryManagementSystem {
  Property view : String = "runtime";
  Property architectureStyle : String = "Client server";
  Property source : String = "AI";
  Component FaceDetectionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Detect the user's face using the YOLOv5 model, contours detection";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component GestureRecognitionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Recognize gestures using threading for concurrent execution";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component HandTrackingModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Track the user's hand using Background Subtraction (BS).";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component InteractionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Support single-hand interaction and exclude multiple-hand interaction";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component ActionCustomizationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Allow users to customize actions for specific gestures using a gesture library";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component CollaborationModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Enable real-time collaboration and support interactions on the same screen";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Component LatencyReductionModule = new Component {
    Port in;
    Port out;
    Property responsibility : String = "Analyze gestures and reflect the corresponding action on the screen";
    Property state : String = "stateless";
    Property scaling : String = "horizontal";
  }
  Attachment FaceDetectionModule.out to GestureRecognitionModule.in {
    Property delivery : String = "at-least-once";
    Property ordering : Boolean = false;
    Property backpressure : String = "bounded";
  }
}

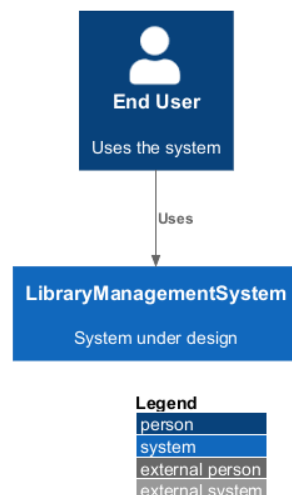
```

```

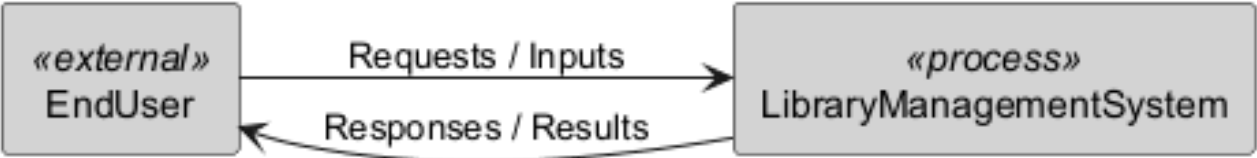
Attachment GestureRecognitionModule.out to HandTrackingModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = true;
  Property backpressure : String = "bounded";
}
Attachment HandTrackingModule.out to InteractionModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = false;
  Property backpressure : String = "bounded";
}
Attachment InteractionModule.out to ActionCustomizationModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = false;
  Property backpressure : String = "bounded";
}
Attachment ActionCustomizationModule.out to CollaborationModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = false;
  Property backpressure : String = "bounded";
}
Attachment CollaborationModule.out to LatencyReductionModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = false;
  Property backpressure : String = "bounded";
}
Attachment LatencyReductionModule.out to InteractionModule.in {
  Property delivery : String = "at-least-once";
  Property ordering : Boolean = true;
  Property backpressure : String = "bounded";
}
Property resilience_pattern : String = "Retry, CircuitBreaker";
Property delivery : String = "at-least-once";
Property ordering : Boolean = false;
Property authentication : String = "service-to-service";
Property authorization : String = "RBAC";
}

```

3. Context View



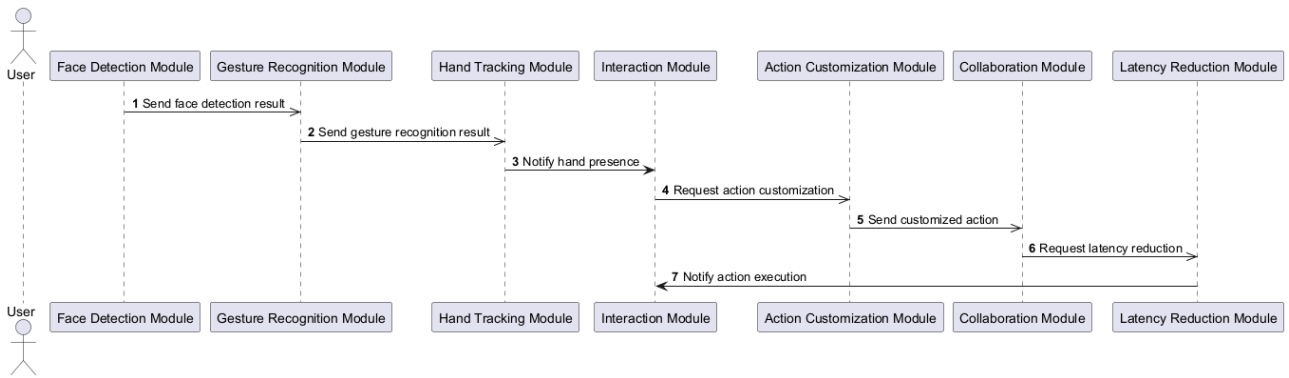
3.1 Data Context View (Level 0 DFD)



4. Logical View (C4 Container Diagram)



5. Process View (Runtime Interaction)



6. Physical View (Deployment Diagram)

